Software Requirements Specification

Railway Ticket Reservation System

Version 2.0 | Revised 12 November 2026 | Team 15 | UE24CS341A

This revision specifies the delivered C++ application and its verification evidence. It aligns the submission documents with the implementation and identifies the remaining team acceptance actions. It is prepared for review; signatures and acceptance are not implied.

Team member	USN	Role
Deeptanshu Kumar	PES1UG24AM357	Developer
Amar	PES1UG24AM340	Tester
Aaryaprasad	PES1UG24AM331	Product Owner

Revision history

Version	Date	Change
1.0	6 September 2026	Original submitted baseline; retained in repository history
2.0	12 November 2026	Reconcile actual stack, interfaces, policies, traceability and current verification evidence

Application baseline: 6b5464298c949806ebba0b7bd1d903c962b5255d. The final feature integration is main commit fb1b4d55957ab8022e136396d656bec870c08375, merged by Amar in PR 12 on 3 November 2026. Current evidence was generated on 12 November 2026.

1 Introduction

1.1 Purpose and audience

The system lets passengers search demonstration train records, inspect fares and inventory, book tickets, view history and cancel eligible reservations. This specification is the requirements baseline for developers, tester, product owner, mentor and evaluator. Each requirement has an identifier, measurable acceptance condition and test reference.

1.2 Scope

The academic web application covers passenger accounts, search, train schedules, availability, passenger entry, confirmation, unique PNR, ticket/history, cancellation/refund recording, administrator train maintenance and booking/cancellation audit events. Real railway operations, live timetable integration and payment settlement are excluded.

1.3 Definitions and references

Term	Meaning
FR / NFR	Functional / Non-Functional Requirement
PNR	Random ten-digit booking reference
RTM	Requirements Traceability Matrix
RBAC	Role-based access control
Opaque session	Random bearer token resolved through server-side storage
UTC	Time basis used by schedule/cancellation calculations

- Course Mini Project Deliverables Parts 1, 2 and 3; evaluation checklist; final demo and viva FAQs.
- Repository README, CMakeLists.txt, src/api, src/services, src/db and tests at the application baseline above.
- IEEE 830 style requirements structure; IEEE 1016 style software design description; IEEE 829 style test planning structure, adapted to the supplied course templates.

- docs/SUBMISSION_INDEX.md for the deliverable inventory; docs/DEVIATIONS.md for baseline differences; docs/reports for actual results.

2 Overall description

2.1 Product perspective

A browser client built with HTML, CSS and JavaScript calls a C++17 JSON REST server. Business services persist data through prepared SQLite statements and transactions. The server serves static assets and can terminate TLS directly. This replaces the original SAD React/Node/Express/MySQL/JWT examples without changing the central reservation use cases.

2.2 Roles and functions

Role	Capabilities
Passenger	Register/login/logout; profile; search/schedule/fare/availability; own booking/ticket/history/cancellation
Administrator	Passenger capabilities plus add/update/deactivate trains and read booking audit records
Tester and Product Owner	Verify acceptance, track defects and approve documented policy/design changes

2.3 Environment and constraints

- Server: C++17 compiler, CMake 3.20 or newer, OpenSSL and SQLite amalgamation; Linux build and Docker packaging provided. Client: a modern JavaScript-enabled browser.
- Inventory and fares are whole train/date/class values. There is no segment-level seat reuse or distance pricing; booking does not store boarding/destination stations.
- Dates must be real YYYY-MM-DD values. There is no operating-day calendar, and the backend currently permits past dates; the demonstration uses future dates.
- Default policy is 24 hours before the current origin departure and 90 percent of stored fare, rounded down to whole rupees. Departure/day offsets are interpreted as UTC. These project defaults require genuine policy acceptance.
- Preview does not hold seats. Confirmation rechecks current inventory and fare transactionally. Client retries have no idempotency key and may create another booking if capacity remains.
- Refund amounts are recorded, not transferred. No payment state machine or production railway integration is delivered.

3 External interface requirements

3.1 User interface

Provide labelled registration/login/profile forms; search criteria and filters; results with train number/name, departure/arrival, duration and starting fare; details with schedule and class availability; passenger form, preview, ticket, history and cancellation confirmation. Admin screens maintain trains. Validation messages identify fields and navigation remains usable at desktop and mobile sizes. Availability is shown in train details rather than directly on every search card, a documented baseline difference.

3.2 Hardware and software interfaces

No dedicated railway hardware is required. The browser uses JSON over HTTP/HTTPS. The C++ process accesses one local SQLite database file. OpenSSL supplies password hashing, secure randomness and optional TLS. Environment variables select database, port, web directory, policy and TLS files. Administrator promotion is a local CLI operation, not public registration.

3.3 Communication

Credentials are submitted in JSON bodies and bearer tokens in Authorization headers. Protected requests resolve identity from the session, never a user ID supplied by the client. Deployed transport must use

configured TLS or a controlled TLS-terminating proxy. Secrets are excluded from URLs, logs and error messages.

4 Functional requirements

RTS-F-001 Passenger registration

The system shall register a passenger using name, email, phone and password, normalize email, and reject duplicate email or invalid fields without creating an account.

Acceptance: POST /api/auth/register returns 201 for valid data, 409 for duplicate email, and 400 for invalid data; no password or hash is returned.

Priority: High. Traceability: TC-REG-01, TC-REG-02. Module: UserService / Routes.

RTS-F-002 Authentication and session lifecycle

The system shall authenticate valid credentials, issue an opaque bearer session, reject invalid credentials, expire sessions after 24 hours, and invalidate the current session on logout.

Acceptance: Valid login returns 200 and token; wrong credentials return generic 401; five failures trigger a 300-second lock (429); expired or logged-out tokens fail protected requests.

Priority: High. Traceability: TC-AUTH-01, TC-AUTH-02, TC-AUTH-03, TC-AUTH-04. Module: AuthService / Routes.

RTS-F-003 Train search

The system shall search active trains by source, destination and date, requiring source to precede destination in the stored stop sequence.

Acceptance: GET search uses date=YYYY-MM-DD, returns 200 with matching trains or an empty list; invalid criteria return 400; class, fare, time and sort filters obey documented inputs.

Priority: High. Traceability: TC-SEARCH-01, TC-SEARCH-03, TC-SEARCH-04. Module: TrainService / Routes.

RTS-F-004 Train details

The system shall display train number/name, requested route, departure, arrival, duration and fare, with ordered schedule and class fares available for inspection.

Acceptance: Search summary, schedule and fares endpoints agree with fixture values; unknown train returns 404.

Priority: High. Traceability: TC-SEARCH-02. Module: TrainService / web/app.js.

RTS-F-005 Seat availability

The system shall return class-wise current available seats for a train and date.

Acceptance: For each class, availableSeats equals totalSeats minus sum of seat_reservations.seats for the same train/date/class; reservations on another date do not reduce it.

Priority: High. Traceability: TC-AVAIL-01, TC-AVAIL-02. Module: TrainService / seat_reservations.

RTS-F-006 Passenger booking form

The system shall let an authenticated passenger select a train, date and class, enter one to six passengers and obtain a fare/availability preview before confirmation.

Acceptance: Valid preview returns 200 with passengerCount, farePerPassenger, totalFare and availableSeats, and writes no booking; unauthenticated preview returns 401.

Priority: High. Traceability: TC-BOOK-01. Module: BookingService / web/app.js.

RTS-F-007 Passenger validation

The system shall validate all passenger fields before preview and confirmation.

Acceptance: Name is 2–60 permitted characters; age is an integer 1–120; gender is M/F/O; zero or more than six passengers is rejected; indexed errors identify the failing passenger and no partial booking is written.

Priority: High. Traceability: TC-BOOK-02, TC-BOOK-06. Module: BookingService / Validation.

RTS-F-008 Final availability check

The system shall recheck active train, class and available capacity inside the confirmation transaction.

Acceptance: If seats were consumed after preview, confirmation returns 409 and creates no booking, passenger, inventory or success audit rows.

Priority: High. Traceability: TC-BOOK-03. Module: BookingService / Database.

RTS-F-009 Unique PNR

The system shall create a unique ten-digit nonsequential PNR for every successfully confirmed reservation.

Acceptance: Two successful bookings have distinct ten-digit PNR strings persisted under a UNIQUE database constraint; PNR lookup retrieves the owner ticket.

Priority: High. Traceability: TC-BOOK-04. Module: BookingService / bookings.

RTS-F-010 Atomic seat reservation

The system shall allocate one distinct seat per passenger and reduce available inventory by exactly the passenger count on successful confirmation.

Acceptance: For n passengers, exactly n seats and one linked quantity-n reservation are committed; availability drops by n once, and failed confirmation leaves it unchanged.

Priority: High. Traceability: TC-BOOK-05, TC-CONC-01. Module: BookingService / Database.

RTS-F-011 Ticket confirmation

The system shall display confirmed booking reference, train, travel date, class, passengers, seat numbers and fare.

Acceptance: POST booking returns 201; authenticated GET /api/bookings/{pnr} returns 200 with the same ticket; unknown or another owner PNR returns 404.

Priority: High. Traceability: TC-TICKET-01. Module: BookingService / web/app.js.

RTS-F-012 Booking history

The system shall display the authenticated passenger’s confirmed and cancelled bookings in newest-first order.

Acceptance: GET /api/bookings returns only the current user’s records including ticket details; a new account has an empty list.

Priority: Medium. Traceability: TC-HISTORY-01. Module: BookingService / Routes.

RTS-F-013 Eligible cancellation

The system shall permit the owner to cancel a confirmed booking when the configured cutoff is satisfied, and reject duplicate or late cancellation.

Acceptance: Default cutoff is 24 hours before the first stored departure interpreted in UTC including first-stop day offset; below cutoff or already cancelled returns 409; successful cancellation returns 200 and CANCELLED.

Priority: High. Traceability: TC-CANCEL-01, TC-CANCEL-04. Module: BookingService.

RTS-F-014 Refund quotation and recording

The system shall display the eligible refund before confirmation and persist the calculated refund on successful cancellation.

Acceptance: Read-only quote shows eligibility and policy; default refund is floor(totalFare × 90 / 100) integer rupees; configured 0–100 percent is honored; this is a record, not fund settlement.

Priority: High. Traceability: TC-CANCEL-02. Module: BookingService.

RTS-F-015 Seat restoration

The system shall restore only the cancelled booking's seats exactly once within the cancellation transaction.

Acceptance: Linked reservation is removed; availability increases by passengerCount; second cancellation does not increase it; released seats can be reused without active duplicates; failure rolls back refund/status/inventory/audit.

Priority: High. Traceability: TC-CANCEL-03, TC-CANCEL-05. Module: BookingService / Database.

RTS-F-016 Booking ownership

The system shall prevent a passenger from viewing, quoting or cancelling another passenger's booking, irrespective of supplied owner fields.

Acceptance: Other-owner requests return 404; missing/invalid/revoked bearer sessions return 401; cancellation service checks the authenticated owner inside its transaction.

Priority: High. Traceability: TC-SEC-03. Module: Authorization / BookingService.

RTS-F-017 Administrative train management

The system shall allow active administrator sessions to create, update and deactivate train, ordered stop and class/fare/capacity data.

Acceptance: Passenger access is 403 and unauthenticated access 401; valid changes appear in search/details; deactivation hides new bookings while old tickets/cancellation remain accessible; invalid/failed changes roll back.

Priority: High. Traceability: TC-ADMIN-01, TC-ADMIN-02, TC-ADMIN-03, TC-ADMIN-04. Module: AdminTrainService / AdminRoutes.

RTS-F-018 Booking audit events

The system shall persist timestamped booking-created and booking-cancelled events with actor, booking, train and PNR identifiers.

Acceptance: One matching event is committed with each successful transition; failed transactions leave no success event; authorized admin can retrieve events.

Priority: Medium. Traceability: TC-AUDIT-01, TC-AUDIT-03. Module: AuditService / audit_events.

5 Non Functional Requirements

RTS-NF-001 Response time

Search and normal booking shall provide a user-visible response within 3 seconds for at least 90% of requests under normal project load.

Acceptance: Measure search and booking separately; each p90 ≤3 seconds. Local runner: 20 search and 20 booking requests at four clients; browser rendering is separately inspected and the local result is not production capacity certification.

Priority: High. Traceability: TC-PERF-01. Module: TrainService / BookingService / browser.

RTS-NF-002 Concurrent integrity

The system shall prevent allocation of one train/date/class seat to two concurrent confirmed bookings.

Acceptance: Simultaneous reservation tests show no duplicate active seat and no overselling; direct duplicate inserts are rejected by the seat trigger.

Priority: High. Traceability: TC-CONC-01, TC-CONC-02. Module: Database / BookingService.

RTS-NF-003 Safe errors

The system shall return clear validation and safe error messages without database internals, stack traces or credentials.

Acceptance: Invalid JSON, wrong types, oversized bodies/IDs and simulated database failures return controlled errors with no SQL/stack/secret disclosure.

Priority: High. Traceability: TC-ERR-01, TC-ERR-02. Module: Routes / Validation.

RTS-NF-004 Availability and recovery

The system shall support normal project use and recover gracefully from temporary application/database failures while retaining consistent committed data.

Acceptance: Restart after clean stop and abrupt process termination preserves committed tickets/inventory/audit; SQLite integrity check is ok; rollback tests retain consistent data. Unrecoverable media corruption is outside this acceptance test.

Priority: Medium. Traceability: TC-REC-01, TC-CANCEL-05. Module: Database / executable.

RTS-NF-005 Password protection

The system shall persist passwords only as salted secure one-way hashes.

Acceptance: Stored PBKDF2-HMAC-SHA256 values differ for identical passwords; production default 600000 iterations; no plaintext password is persisted or returned.

Priority: High. Traceability: TC-SEC-01. Module: PasswordHasher / UserService.

RTS-NF-006 Role enforcement

The system shall restrict all administrator functions to currently authorized administrator sessions.

Acceptance: Passenger/anonymous/revoked sessions are rejected; role is reloaded from database on every authenticated request; public registration cannot choose ADMIN.

Priority: High. Traceability: TC-SEC-02, TC-ADMIN-02. Module: Authorization / AuthService.

RTS-NF-007 Responsive user interface

The system shall provide a readable, usable core booking flow on common desktop and mobile viewports.

Acceptance: At 390×844 and 1366×900, controls/text remain reachable and no horizontal page overflow prevents use; browser evidence must identify each flow actually exercised on each viewport.

Priority: Medium. Traceability: TC-UX-01. Module: web/index.html / style.css / app.js.

RTS-NF-008 Auditability

The system shall maintain persistent auditable timestamped records of successful booking and cancellation actions.

Acceptance: Events can be correlated to ticket/actor and retrieved through admin audit; preserved after restart; failed actions have no success event.

Priority: Medium. Traceability: TC-AUDIT-01, TC-AUDIT-03, TC-REC-01. Module: AuditService / Database.

5.1 Security objectives

- SO-1 Protect passenger accounts and reservation information against unauthorized disclosure and modification.
- SO-2 Preserve the integrity of reservation, cancellation and inventory state under invalid and concurrent requests.

5.2 Security requirements

RTS-SR-001 One-way password storage

The system shall never store plaintext passwords and shall use salted PBKDF2-HMAC-SHA256 with bounded iteration parsing and constant-time derived-hash comparison.

Acceptance: Hash inspection and verification tests pass; identical passwords have distinct salts; malformed or implausible stored iteration counts fail safely; weaker hashes upgrade after successful login.

Priority: High. Traceability: TC-SEC-01, TC-SEC-07. Module: PasswordHasher / AuthService.

RTS-SR-002 Authentication and authorization

The system shall enforce authentication, current role and booking ownership for every protected operation.

Acceptance: Anonymous/invalid sessions fail with 401; passenger admin attempts fail with 403; foreign bookings are hidden with 404; session hashes only are persisted and logout revokes access.

Priority: High. Traceability: TC-AUTH-03, TC-SEC-02, TC-SEC-03. Module: AuthService / Authorization.

RTS-SR-003 Secure communication

The system shall support HTTPS/TLS for credential and reservation exchange in a supported deployment and validate it over a live connection.

Acceptance: Real executable with certificate/key completes trusted local TLS handshake and authenticated HTTPS workflow; incomplete TLS configuration fails startup. Plain HTTP development mode is not evidence of compliance for a hosted deployment.

Priority: High. Traceability: TC-SEC-04. Module: TlsConfig / main / cpp-httplib.

RTS-SR-004 Input safety

The system shall validate untrusted input, use parameterized SQL and safely render user-derived content to reduce injection risks.

Acceptance: SQL/script/control-character/wrong-type/oversized inputs cannot alter schema or execute unintended scripts; invalid data produces controlled errors and no unintended writes.

Priority: High. Traceability: TC-SEC-05, TC-BOOK-02, TC-ERR-02. Module: Validation / Database / web/app.js.

RTS-SR-005 Secret-safe diagnostics

The system shall exclude passwords and bearer authentication values from logs, URLs and error messages.

Acceptance: Access logs include method/path/status while dropping query values; secret redaction tests and error/URL inspections show no submitted password or raw token.

Priority: High. Traceability: TC-SEC-06, TC-AUDIT-02. Module: Log / Routes / web/app.js.

6 Quality attributes and acceptance

Correctness is evaluated through inventory/PNR/refund invariants; reliability through rollback and restart scenarios; security through credential, role, owner, TLS and input checks; usability through selected browser viewports. All high-priority requirements need passing evidence or an explicit accepted disposition. No critical unresolved defect should block acceptance. The current automated reports do not substitute for team UAT or signatures.

Evidence	Current result and limit
Unit and API suite	201 tests across 49 suites pass; not 201 distinct STP cases
System validation	122 assertions pass; small local dataset, real subprocess HTTP/TLS and restart checks
Coverage	94.7% lines and 46.7% branches in C++ src excluding main.cpp; not browser/deployment coverage
Responsive UI	Selected 1366×900 desktop and 390×844 mobile observations; physical-device/cross-browser acceptance not established

7 System models

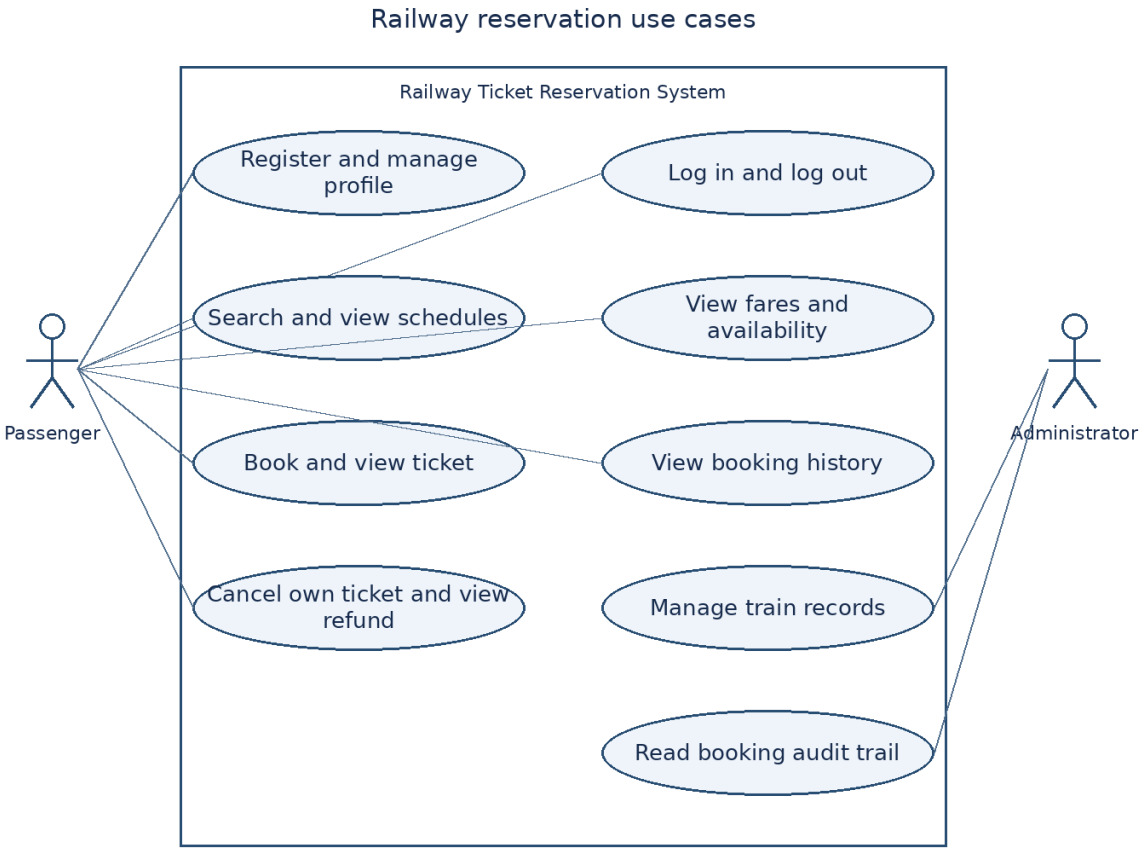


Figure 1 Passenger and administrator use cases

Passenger operations require authentication where they access private records. Administrator operations add a current database role check. Booking/cancellation service rules enforce inventory consistency and ownership independently of UI controls.

8 Requirements traceability

Requirement	Module	Test case references
RTS-F-001	UserService / Routes	TC-REG-01, TC-REG-02
RTS-F-002	AuthService / Routes	TC-AUTH-01, TC-AUTH-02, TC-AUTH-03, TC-AUTH-04
RTS-F-003	TrainService / Routes	TC-SEARCH-01, TC-SEARCH-03, TC-SEARCH-04
RTS-F-004	TrainService / web/app.js	TC-SEARCH-02
RTS-F-005	TrainService / seat_reservations	TC-AVAIL-01, TC-AVAIL-02
RTS-F-006	BookingService / web/app.js	TC-BOOK-01
RTS-F-007	BookingService / Validation	TC-BOOK-02, TC-BOOK-06

Requirement	Module	Test case references
RTS-F-008	BookingService / Database	TC-BOOK-03
RTS-F-009	BookingService / bookings	TC-BOOK-04
RTS-F-010	BookingService / Database	TC-BOOK-05, TC-CONC-01
RTS-F-011	BookingService / web/app.js	TC-TICKET-01
RTS-F-012	BookingService / Routes	TC-HISTORY-01
RTS-F-013	BookingService	TC-CANCEL-01, TC-CANCEL-04
RTS-F-014	BookingService	TC-CANCEL-02
RTS-F-015	BookingService / Database	TC-CANCEL-03, TC-CANCEL-05
RTS-F-016	Authorization / BookingService	TC-SEC-03
RTS-F-017	AdminTrainService / AdminRoutes	TC-ADMIN-01, TC-ADMIN-02, TC-ADMIN-03, TC-ADMIN-04
RTS-F-018	AuditService / audit_events	TC-AUDIT-01, TC-AUDIT-03
RTS-NF-001	TrainService / BookingService / browser	TC-PERF-01
RTS-NF-002	Database / BookingService	TC-CONC-01, TC-CONC-02
RTS-NF-003	Routes / Validation	TC-ERR-01, TC-ERR-02
RTS-NF-004	Database / executable	TC-REC-01, TC-CANCEL-05
RTS-NF-005	PasswordHasher / UserService	TC-SEC-01
RTS-NF-006	Authorization / AuthService	TC-SEC-02, TC-ADMIN-02
RTS-NF-007	web/index.html / style.css / app.js	TC-UX-01
RTS-NF-008	AuditService / Database	TC-AUDIT-01, TC-AUDIT-03, TC-REC-01
RTS-SR-001	PasswordHasher / AuthService	TC-SEC-01, TC-SEC-07
RTS-SR-002	AuthService / Authorization	TC-AUTH-03, TC-SEC-02, TC-SEC-03
RTS-SR-003	TlsConfig / main / cpp-httplib	TC-SEC-04
RTS-SR-004	Validation / Database / web/app.js	TC-SEC-05, TC-BOOK-02, TC-ERR-02
RTS-SR-005	Log / Routes / web/app.js	TC-SEC-06, TC-AUDIT-02

Named automated tests are listed in the Test Plan and source/test_cases.json. Passing unit/API checks, system assertions and selected browser observations are distinct evidence types. Human UAT and PO/mentor acceptance remain pending; the matrix is not a fabricated sign-off.

9 Open acceptance actions

- Tester reviews reports, runs final human acceptance and uploads the two sprint videos through the designated friend.
- Product Owner reviews policy defaults, scope limitations and stack/interface deviations.
- Connect current Jira to reconcile the dated 13 October snapshot with later dates/statuses; GitHub course milestones remain separate from original Jira sprints.

Reviewer	Review responsibility	Signature and actual date
Amar	Test Plan and verification review	Pending
Aaryaprasad	Requirements and policy acceptance	Pending
Rajeshwari Adarakatti	Mentor evaluation	Pending